Full Stack Engineering Assessment

Secure Remediation Workspace

Build a small application that allows organizations to assign remediation work, track progress, and approve completion.

Time and tools

- Time limit: Eight hours, including documentation. Stop at the limit and explain unfinished work.
- AI tools: Coding agents and assistants are allowed. You are responsible for understanding and verifying everything submitted.
- Stack: TypeScript, React or Next.js, a Node.js backend, and PostgreSQL or MongoDB.
- Delivery: A repository or source archive that runs locally. No paid services or deployment required.

Prioritize a reliable end-to-end implementation. Additional features do not earn extra credit.

Organizations and permissions

Seed two organizations, each containing at least two managers and two members. A user belongs to one organization.

Role	Permissions within their organization
Manager	View all remediation items; create, edit, assign, and reassign items; review submitted work
Member	View only their assigned items; start work and submit completion

Enforce these rules on the server:

- Users cannot read or modify another organization's data or assign items to its users.
- Managers cannot approve or request changes on their own assigned items.
- Reassignment immediately removes the previous member's access.
- Lists, counts, detail views, and mutations follow the same authorization rules. Hiding a button is not an authorization control.

Authentication and session security

Implement login and logout using seeded accounts. You may use established authentication libraries; do not implement cryptographic primitives yourself. Use signed access tokens with a clearly documented session lifecycle.

Your implementation must:

- Store passwords using an appropriate password-hashing algorithm.
- Validate token signatures, explicitly allowed algorithms, expiration, issuer, and audience.
- Reject malformed, tampered, expired, or otherwise invalid tokens.
- Establish identity from validated authentication data. Never trust a caller-supplied user ID, organization ID, or role as proof of identity or authorization.
- Explain where tokens are stored and how that choice addresses XSS and CSRF risks.
- Define and implement logout behavior, including what happens to previously issued tokens.
- Ensure a disabled user or revoked session cannot continue making authenticated requests.
- Protect login against repeated attempts and return errors that do not reveal whether an account exists.
- Keep credentials, signing keys, passwords, and tokens out of source control and logs.

For this exercise, a short-lived access token with a server-side session or revocation check is sufficient. Refresh tokens are optional; if implemented, explain rotation, revocation, and reuse handling.

Registration, password reset, MFA, and SSO are out of scope. Briefly explain how you would add them later.

Backend integrity and security

- Validate inputs and explicitly restrict writable fields. Prevent clients from directly modifying organization, actor, role, audit, or workflow-controlled fields.
- Persist audit events for creation, edits, reassignment, and transitions. Record actor, timestamp, and relevant changes. Clients must not directly edit audit events.
- Handle stale edits and competing actions safely. Ensure item changes and audit events remain consistent if an operation fails.
- Bound queries and page sizes; define indexes appropriate to your actual queries.
- Return useful errors without exposing stack traces, secrets, or sensitive internal details. Configure browser-origin and cookie protections appropriately.

Document remaining limitations rather than claiming production readiness without evidence.

Remediation workflow

Each item includes a title, plain-text description, assignee, due date, status, completion note, and relevant identifiers and timestamps.

Implement these transitions:

Current status	Action	Authorized user	New status
Open	Start work	Current assignee	In progress
In progress	Submit a non-empty completion note	Current assignee	Pending review
Pending review	Approve	Manager other than the assignee	Closed
Pending review	Request changes with a non-empty reason	Manager other than the assignee	In progress

Managers may edit or reassign items only while Open or In progress. Reassignment preserves status. Closed items are read-only.

Define whether due dates are calendar dates or timestamps and apply that definition consistently. An item is overdue when its due date has passed and it is not Closed. Deletion and reopening are out of scope.

Frontend experience

Provide:

- Login and logout.
- An inbox with server-side pagination, status and overdue filters.
- A manager form for creating and editing items.
- An item detail view with permitted actions and activity history.
- Clear loading, empty, validation, failure, and session-expiration states.
- Input preservation after recoverable failures and protection against accidental duplicate submissions.
- Labeled controls, keyboard usability, and a layout usable on smaller screens.

Use a UI library if helpful. We value clarity and usability over custom visual styling.

Automated tests

Include meaningful tests covering:

1. Successful login and an authenticated request.
2. Invalid credentials and malformed, tampered, expired, wrong-issuer, and wrong-audience tokens.
3. Logout or session revocation preventing subsequent access.
4. A disabled user being denied access.
5. Cross-organization access attempts against reads and mutations.
6. An unauthorized action within the same organization.
7. A valid workflow and an invalid transition.
8. A stale edit or competing review action, checking persisted state and audit consistency.
9. One frontend interaction involving validation, submission failure, or session expiration.

Use a real test database for authorization and persistence integration tests. Mocking the database alone does not demonstrate those guarantees.

Provide repeatable test commands and isolated test data. We value useful assertions over coverage percentages.

Architecture and operational judgment

Write a short note explaining:

- Your application structure and data model.
- Authentication, token validation, session revocation, and authorization boundaries.
- Concurrency and audit consistency.
- Important indexes and the queries they support.
- How you would evolve the application for 500 organizations, five million items, and bursts of 100 updates per second.
- How you would investigate failures without logging sensitive customer data.
- How you would introduce a database change safely.

State assumptions and discuss what you would measure before adding infrastructure. You do not need to implement the scaled architecture.

Submission

Include:

- Runnable source, dependency lockfile, database setup, and repeatable seed data.
- An example environment file without real secrets.
- Setup instructions, demo accounts, and test commands.
- Time spent, completed scope, assumptions, and known limitations.
- Your next three priorities if given another day.
- The architecture note.
- A brief handoff for the next engineer.
- An AI usage note describing tools used and concrete examples of how you verified generated work. Include a suggestion you corrected or rejected, if applicable. Full conversation histories are not required.

If you did not use AI, describe your verification approach instead.

Follow up session

We will review the application with you, trace a request through the system, run selected tests, discuss security and scaling decisions, and ask for one small modification.

Coding agents remain allowed. You should be able to explain the implementation, identify its limitations, and verify your changes.

We evaluate correctness, security, testing, maintainability, user experience, practical system design, and ownership—not typing speed, code volume, or architectural complexity.